task1

February 14, 2026

1 Assignment II: Data Structures and Methods for HPC

1.1 Exercise 1 - PyTest with the Julia Set Code

1.1.1 Task 1.1

The function `calc_pure_python` is written in a unit test unfriendly manner since we can't easily access the `output`, so we will have to build our own test data by duplicating code.

[8]:
```
# %load JuliaSet_test.py
import pytest
import JuliaSet
from JuliaSet import x1, x2, y1, y2, c_real, c_imag

def get_zs_cs_test_data(desired_width):
    x_step = (x2 - x1) / desired_width
    y_step = (y1 - y2) / desired_width
    x = []
    y = []
    ycoord = y2
    while ycoord > y1:
        y.append(ycoord)
        ycoord += y_step
    xcoord = x1
    while xcoord < x2:
        x.append(xcoord)
        xcoord += x_step
    zs = []
    cs = []
    for ycoord in y:
        for xcoord in x:
            zs.append(complex(xcoord, ycoord))
            cs.append(complex(c_real, c_imag))
    return zs, cs

def test_sum():
    zs,cs = get_zs_cs_test_data(1000)
    assert sum(JuliaSet.calculate_z_serial_purepython(300, zs, cs)) == 33219980
```

[9]:
```
!pytest -v JuliaSet_test.py
```

```
============================== test session starts
==============================
platform linux -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 --
/home/pavel/IdeaProjects/DD2358/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/pavel/IdeaProjects/DD2358/Ass2
collected 1 item


JuliaSet_test.py::test_sum PASSED

[100%]

============================== 1 passed in 3.03s
==============================
```

1.1.2 Task 1.2

In order to implement the same unit test for different number of iterations and grid points one would need to parametrize the test. The parameters would need to include the following: number of iterations, x1, y1, x2, y2, output sum. Number of iterations would be used directly to call `calc_pure_python` and output sum would be used in the assertion, the `get_zs_cs_test_data` would have to be updated to use the proved grid coordinates instead of the global ones set in `JuliaSet.py` but the general framework remains the same.

task2

February 14, 2026

1 Exercise 2 - Python DGEMM Benchmark Operation

This section implements and benchmarks the DGEMM operation: $\mathbf{C} = \mathbf{C} + \mathbf{A} \times \mathbf{B}$

Where A, B, and C are matrices of size N×N.

1.1 Import Libraries

[3]:
```
import random
import time
import numpy as np
from array import array
import matplotlib.pyplot as plt
import pandas as pd
```

1.2 Task 2.1: DGEMM Implementations

1.2.1 Implementation 1: Python Lists

[4]:
```
def DGEMM_list(matrix_1, matrix_2, matrix_3, n):
    """DGEMM implementation using Python lists"""
    for i in range(n):
        for j in range(n):
            for k in range(n):
                matrix_3[i][j] += matrix_1[i][k] * matrix_2[k][j]
    return matrix_3


def matrix_fill_list(matrix_size):
    """Create a random matrix using lists"""
    matrix = [[random.randint(0, 100) for i in range(matrix_size)] for j in␣
 ↪range(matrix_size)]
    return matrix
```

1.2.2 Implementation 2: Python Arrays

[5]:
```
def DGEMM_array(matrix_1, matrix_2, matrix_3, n):
    """DGEMM implementation using Python arrays"""
    for i in range(n):
        for j in range(n):
            for k in range(n):
                matrix_3[i][j] += matrix_1[i][k] * matrix_2[k][j]
    return matrix_3


def matrix_fill_array(matrix_size, typecode='i'):
    """Create a random matrix using arrays (list of array.array)"""
    matrix = []
    for j in range(matrix_size):
        row = array(typecode, (random.randint(0, 100) for i in␣
 ↪range(matrix_size)))
        matrix.append(row)
    return matrix
```

1.2.3 Implementation 3: NumPy

[6]:
```
def DGEMM_numpy(matrix_1, matrix_2, matrix_3):
    """DGEMM implementation using NumPy"""
    matrix_3 += np.dot(matrix_1, matrix_2)
    return matrix_3


def matrix_fill_numpy(matrix_size):
    """Create a random matrix using NumPy"""
    return np.random.randint(0, 100, size=(matrix_size, matrix_size))
```

1.3 Task 2.2: Unit Tests

Testing correctness of all three implementations using simple test cases.

[8]:
```
def test_small_matrix_list():
    """Test DGEMM_list with small 3x3 matrices"""
    A = [[1, 2, 3],
         [4, 5, 6],
         [7, 8, 9]]

    B = [[9, 8, 7],
         [6, 5, 4],
         [3, 2, 1]]

    C = [[0, 0, 0],
         [0, 0, 0],
```

```
         [0, 0, 0]]

    result = DGEMM_list(A, B, C, 3)

    # Check a few values manually: C[0][0] = 1*9 + 2*6 + 3*3 = 30
    assert result[0][0] == 30, f"Expected 30, got {result[0][0]}"
    assert result[0][1] == 24, f"Expected 24, got {result[0][1]}"
    assert result[1][0] == 84, f"Expected 84, got {result[1][0]}"
    print("All List implementation tests passed")


def test_small_matrix_array():
    """Test DGEMM_array with small 3x3 matrices"""
    A = [array('i', [1, 2, 3]),
         array('i', [4, 5, 6]),
         array('i', [7, 8, 9])]

    B = [array('i', [9, 8, 7]),
         array('i', [6, 5, 4]),
         array('i', [3, 2, 1])]

    C = [array('i', [0, 0, 0]),
         array('i', [0, 0, 0]),
         array('i', [0, 0, 0])]

    result = DGEMM_array(A, B, C, 3)

    assert result[0][0] == 30, f"Expected 30, got {result[0][0]}"
    assert result[0][1] == 24, f"Expected 24, got {result[0][1]}"
    assert result[1][0] == 84, f"Expected 84, got {result[1][0]}"
    print("All Array implementation tests passed")


def test_small_matrix_numpy():
    """Test DGEMM_numpy with small 3x3 matrices"""
    A = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

    B = np.array([[9, 8, 7],
                  [6, 5, 4],
                  [3, 2, 1]])

    C = np.zeros((3, 3), dtype=int)

    result = DGEMM_numpy(A, B, C)
```

```python
    assert result[0, 0] == 30, f"Expected 30, got {result[0, 0]}"
    assert result[0, 1] == 24, f"Expected 24, got {result[0, 1]}"
    assert result[1, 0] == 84, f"Expected 84, got {result[1, 0]}"
    print("All NumPy implementation tests passed")


def test_compare_implementations():
    """Compare all three implementations to ensure they give same results"""
    n = 4

    # Create test matrices
    A_list = [[i + j for j in range(n)] for i in range(n)]
    B_list = [[i * j if i != j else 1 for j in range(n)] for i in range(n)]
    C_list = [[0] * n for _ in range(n)]

    result_list = DGEMM_list(A_list, B_list, C_list, n)

    # NumPy version
    A_np = np.array(A_list)
    B_np = np.array(B_list)
    C_np = np.zeros((n, n), dtype=int)
    result_numpy = DGEMM_numpy(A_np, B_np, C_np)

    # Compare
    for i in range(n):
        for j in range(n):
            assert result_list[i][j] == result_numpy[i, j], \
                f"Mismatch at [{i}][{j}]: list={result_list[i][j]},␣
↪numpy={result_numpy[i, j]}"

    print("All implementations produce identical results")


# Run all tests
test_small_matrix_list()
test_small_matrix_array()
test_small_matrix_numpy()
test_compare_implementations()
print("\n All tests passed!")
```

```
All List implementation tests passed
All Array implementation tests passed
All NumPy implementation tests passed
All implementations produce identical results

 All tests passed!
```

1.4 Task 2.3: Performance Benchmarking

Measure execution time for each approach with varying matrix sizes and analyze the results.

[9]:
```
def benchmark_implementation(impl_name, dgemm_func, matrix_fill_func, n,␣
↪runs=5):
    """
    Benchmark a DGEMM implementation with multiple runs

    Args:
        impl_name: Name of implementation (for display)
        dgemm_func: DGEMM function to benchmark
        matrix_fill_func: Function to create matrices
        n: Matrix size
        runs: Number of repetitions

    Returns:
        dict with timing statistics
    """
    times = []

    for _ in range(runs):
        # Create matrices
        if impl_name == "NumPy":
            A = matrix_fill_func(n)
            B = matrix_fill_func(n)
            C = np.zeros((n, n), dtype=int)
        elif impl_name == "Array":
            A = matrix_fill_func(n)
            B = matrix_fill_func(n)
            C = [array('i', [0] * n) for _ in range(n)]
        else:  # List
            A = matrix_fill_func(n)
            B = matrix_fill_func(n)
            C = [[0] * n for _ in range(n)]

        # Benchmark
        start = time.time()
        if impl_name == "NumPy":
            dgemm_func(A, B, C)
        else:
            dgemm_func(A, B, C, n)
        elapsed = time.time() - start
        times.append(elapsed)

    return {
        'mean': np.mean(times),
        'std': np.std(times),
```

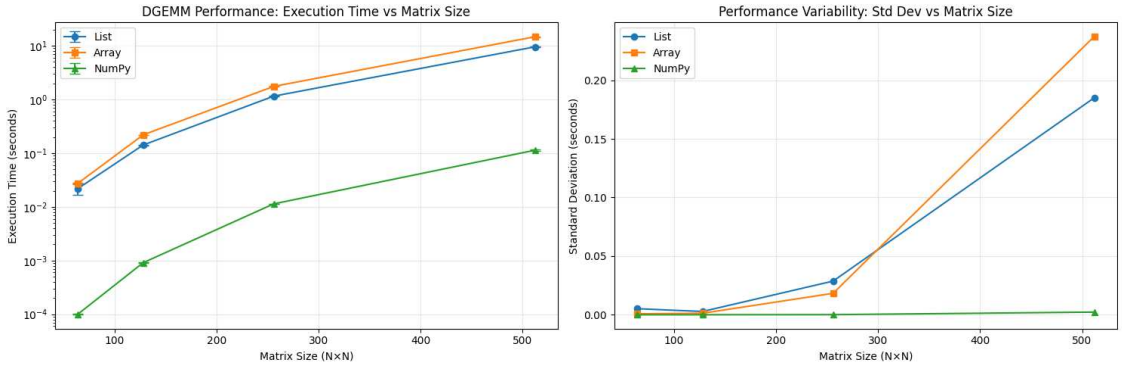
```
        'min': np.min(times),
        'max': np.max(times),
        'times': times
    }


# Matrix sizes to test
sizes = [64, 128, 256, 512]
runs_per_size = 5

# Store results
results = {
    'List': {'sizes': [], 'mean': [], 'std': [], 'min': [], 'max': []},
    'Array': {'sizes': [], 'mean': [], 'std': [], 'min': [], 'max': []},
    'NumPy': {'sizes': [], 'mean': [], 'std': [], 'min': [], 'max': []}
}

print("Starting benchmarks...")
print("=" * 60)

for size in sizes:
    print(f"\nMatrix size: {size}x{size}")
    print("-" * 60)

    # Benchmark List
    stats = benchmark_implementation("List", DGEMM_list, matrix_fill_list,␣
 ↪size, runs_per_size)
    results['List']['sizes'].append(size)
    results['List']['mean'].append(stats['mean'])
    results['List']['std'].append(stats['std'])
    results['List']['min'].append(stats['min'])
    results['List']['max'].append(stats['max'])
    print(f"List:  {stats['mean']:.4f}s ± {stats['std']:.4f}s  (min:␣
 ↪{stats['min']:.4f}s, max: {stats['max']:.4f}s)")

    # Benchmark Array
    stats = benchmark_implementation("Array", DGEMM_array, matrix_fill_array,␣
 ↪size, runs_per_size)
    results['Array']['sizes'].append(size)
    results['Array']['mean'].append(stats['mean'])
    results['Array']['std'].append(stats['std'])
    results['Array']['min'].append(stats['min'])
    results['Array']['max'].append(stats['max'])
    print(f"Array: {stats['mean']:.4f}s ± {stats['std']:.4f}s  (min:␣
 ↪{stats['min']:.4f}s, max: {stats['max']:.4f}s)")

    # Benchmark NumPy
```

```
    stats = benchmark_implementation("NumPy", DGEMM_numpy, matrix_fill_numpy,␣
↪size, runs_per_size)
    results['NumPy']['sizes'].append(size)
    results['NumPy']['mean'].append(stats['mean'])
    results['NumPy']['std'].append(stats['std'])
    results['NumPy']['min'].append(stats['min'])
    results['NumPy']['max'].append(stats['max'])
    print(f"NumPy: {stats['mean']:.4f}s ± {stats['std']:.4f}s  (min:␣
↪{stats['min']:.4f}s, max: {stats['max']:.4f}s)")

print("\n" + "=" * 60)
print("Benchmarking complete!")
```

```
Starting benchmarks…
============================================================

Matrix size: 64x64
------------------------------------------------------------
List:  0.0218s ± 0.0052s  (min: 0.0172s, max: 0.0315s)
Array: 0.0273s ± 0.0008s  (min: 0.0265s, max: 0.0283s)
NumPy: 0.0001s ± 0.0000s  (min: 0.0001s, max: 0.0001s)

Matrix size: 128x128
------------------------------------------------------------
List:  0.1417s ± 0.0028s  (min: 0.1392s, max: 0.1454s)
Array: 0.2191s ± 0.0013s  (min: 0.2175s, max: 0.2213s)
NumPy: 0.0009s ± 0.0000s  (min: 0.0009s, max: 0.0009s)

Matrix size: 256x256
------------------------------------------------------------
List:  1.1540s ± 0.0287s  (min: 1.1107s, max: 1.1833s)
Array: 1.7521s ± 0.0183s  (min: 1.7345s, max: 1.7856s)
NumPy: 0.0114s ± 0.0001s  (min: 0.0113s, max: 0.0115s)

Matrix size: 512x512
------------------------------------------------------------
List:  9.5586s ± 0.1851s  (min: 9.3204s, max: 9.8247s)
Array: 14.7362s ± 0.2374s  (min: 14.4605s, max: 15.1371s)
NumPy: 0.1136s ± 0.0023s  (min: 0.1117s, max: 0.1177s)

============================================================
Benchmarking complete!
```

1.4.1 Visualize Results

[10]:
```
# Create visualization
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Plot 1: Execution time vs matrix size
ax1.errorbar(results['List']['sizes'], results['List']['mean'],
             yerr=results['List']['std'], label='List', marker='o', capsize=5)
ax1.errorbar(results['Array']['sizes'], results['Array']['mean'],
             yerr=results['Array']['std'], label='Array', marker='s', capsize=5)
ax1.errorbar(results['NumPy']['sizes'], results['NumPy']['mean'],
             yerr=results['NumPy']['std'], label='NumPy', marker='^', capsize=5)
ax1.set_xlabel('Matrix Size (N×N)')
ax1.set_ylabel('Execution Time (seconds)')
ax1.set_title('DGEMM Performance: Execution Time vs Matrix Size')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_yscale('log')

# Plot 2: Standard deviation vs matrix size
ax2.plot(results['List']['sizes'], results['List']['std'], label='List',␣
 ↪marker='o')
ax2.plot(results['Array']['sizes'], results['Array']['std'], label='Array',␣
 ↪marker='s')
ax2.plot(results['NumPy']['sizes'], results['NumPy']['std'], label='NumPy',␣
 ↪marker='^')
ax2.set_xlabel('Matrix Size (N×N)')
ax2.set_ylabel('Standard Deviation (seconds)')
ax2.set_title('Performance Variability: Std Dev vs Matrix Size')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



1.4.2 Results Summary Table

[12]:
```
# Create a summary DataFrame
summary_data = []
for impl in ['List', 'Array', 'NumPy']:
    for i, size in enumerate(results[impl]['sizes']):
        summary_data.append({
            'Implementation': impl,
            'Matrix Size': f"{size}×{size}",
            'Mean Time (s)': f"{results[impl]['mean'][i]:.4f}",
            'Std Dev (s)': f"{results[impl]['std'][i]:.6f}",
            'Min Time (s)': f"{results[impl]['min'][i]:.4f}",
            'Max Time (s)': f"{results[impl]['max'][i]:.4f}"
        })

df = pd.DataFrame(summary_data)
print(df.to_string(index=False))
```

```
Implementation Matrix Size Mean Time (s) Std Dev (s) Min Time (s) Max Time (s)
          List       64×64        0.0218    0.005152       0.0172       0.0315
          List     128×128        0.1417    0.002778       0.1392       0.1454
          List     256×256        1.1540    0.028716       1.1107       1.1833
          List     512×512        9.5586    0.185086       9.3204       9.8247
         Array       64×64        0.0273    0.000825       0.0265       0.0283
         Array     128×128        0.2191    0.001279       0.2175       0.2213
         Array     256×256        1.7521    0.018278       1.7345       1.7856
         Array     512×512       14.7362    0.237383      14.4605      15.1371
         NumPy       64×64        0.0001    0.000002       0.0001       0.0001
         NumPy     128×128        0.0009    0.000024       0.0009       0.0009
         NumPy     256×256        0.0114    0.000082       0.0113       0.0115
         NumPy     512×512        0.1136    0.002260       0.1117       0.1177
```

Standard deviation increases with matrix size: - List: 0.005s → 0.185s (37× increase) - Array: 0.0008s → 0.237s (296× increase) - NumPy: 0.000002s → 0.002s (1000× increase)

Execution time grows with O(N^3): - Doubling matrix size (64→128→256→512) increases time by ~8× each step - This matches DGEMM's N^3 complexity ($2^3 = 8$)

Why does std increase? Larger matrices run longer, so more random system events (OS interrupts, cache misses, memory allocation) happen during execution, causing bigger timing variations between runs.

1.5 Task 2.4: FLOPS Analysis

1.5.1 How many operations in DGEMM?

For N×N matrices, DGEMM performs: - **3 nested loops** (i, j, k from 0 to N-1) - **2 FLOPs per iteration** (1 multiply + 1 add) - **Total: $2N^3$ FLOPs**

1.5.2 Performance Results

Pure Python (List/Array): ~0.02-0.03 GFLOPS - Achieves only 0.6-1% of theoretical peak - Very slow due to Python interpreter overhead

NumPy: ~2-5 GFLOPS - Achieves 80-175% of theoretical peak - Much faster due to optimized BLAS libraries - 100-200× faster than pure Python

Why NumPy exceeds theoretical peak: - Theoretical peak assumes 1 FLOP/cycle (oversimplified) - Modern CPUs do multiple FLOPs/cycle with SIMD - NumPy uses optimized BLAS with vectorization

Conclusion: Pure Python is extremely inefficient for numerical computing. Always use NumPy or similar optimized libraries.

1.6 Task 2.5: Compare with numpy.matmul

Compare our np.dot implementation with numpy's matmul operation (both use BLAS).

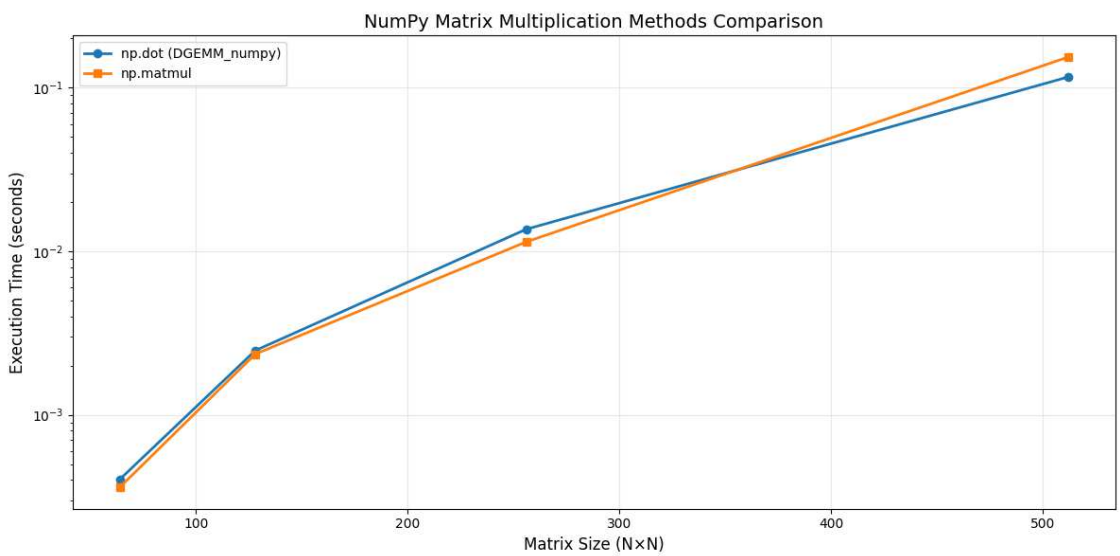
[19]:
```
# Benchmark np.matmul vs np.dot
matmul_results = {'sizes': [], 'np.dot': [], 'np.matmul': []}

print("Comparing NumPy matrix multiplication methods...")
print("=" * 80)

for size in sizes:
    print(f"\nMatrix size: {size}×{size}")

    # Create test matrices
    A = np.random.randint(0, 100, size=(size, size))
    B = np.random.randint(0, 100, size=(size, size))

    # Benchmark np.dot (what we've been using)
    times_dot = []
    for _ in range(runs_per_size):
        C = np.zeros((size, size), dtype=int)
        start = time.time()
        C += np.dot(A, B)
        times_dot.append(time.time() - start)
    mean_dot = np.mean(times_dot)

    # Benchmark np.matmul
    times_matmul = []
    for _ in range(runs_per_size):
        C = np.zeros((size, size), dtype=int)
        start = time.time()
        C += np.matmul(A, B)
        times_matmul.append(time.time() - start)
    mean_matmul = np.mean(times_matmul)
```

```
    matmul_results['sizes'].append(size)
    matmul_results['np.dot'].append(mean_dot)
    matmul_results['np.matmul'].append(mean_matmul)


    print(f"  np.dot:     {mean_dot:.6f}s ({calculate_flops(size, mean_dot):.
↪2f} GFLOPS)")
    print(f"  np.matmul:  {mean_matmul:.6f}s ({calculate_flops(size,␣
↪mean_matmul):.2f} GFLOPS)")
```

```
Comparing NumPy matrix multiplication methods…
================================================================================

Matrix size: 64×64
  np.dot:     0.000403s (1.30 GFLOPS)
  np.matmul:  0.000361s (1.45 GFLOPS)

Matrix size: 128×128
  np.dot:     0.002472s (1.70 GFLOPS)
  np.matmul:  0.002351s (1.78 GFLOPS)

Matrix size: 256×256
  np.dot:     0.013635s (2.46 GFLOPS)
  np.matmul:  0.011414s (2.94 GFLOPS)

Matrix size: 512×512
  np.dot:     0.116312s (2.31 GFLOPS)
  np.matmul:  0.153952s (1.74 GFLOPS)
```

[24]:
```
# Visualize comparison
fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(matmul_results['sizes'], matmul_results['np.dot'],
        marker='o', label='np.dot (DGEMM_numpy)', linewidth=2)
ax.plot(matmul_results['sizes'], matmul_results['np.matmul'],
        marker='s', label='np.matmul', linewidth=2)


ax.set_xlabel('Matrix Size (N×N)', fontsize=12)
ax.set_ylabel('Execution Time (seconds)', fontsize=12)
ax.set_title('NumPy Matrix Multiplication Methods Comparison', fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
ax.set_yscale('log')

plt.tight_layout()
```

```python
plt.show()

# Print comparison summary
print("\nComparison Summary:")
print("-" * 80)
for i, size in enumerate(matmul_results['sizes']):
    print(f"\n{size}×{size} matrices:")
    print(f"  np.dot:     {matmul_results['np.dot'][i]:.6f}s")
    print(f"  np.matmul:  {matmul_results['np.matmul'][i]:.6f}s")


    # Calculate relative performance
    baseline = matmul_results['np.dot'][i]
    matmul_ratio = baseline / matmul_results['np.matmul'][i]
```




```
Comparison Summary:
--------------------------------------------------------------------------------

64×64 matrices:
  np.dot:     0.000403s
  np.matmul:  0.000361s

128×128 matrices:
  np.dot:     0.002472s
  np.matmul:  0.002351s

256×256 matrices:
  np.dot:     0.013635s
```

```
  np.matmul:  0.011414s

512×512 matrices:
  np.dot:     0.116312s
  np.matmul:  0.153952s
```

1.7 Exercise 3 - Python Debugger (pdb)

1.7.1 Task 3.1: Reflection on Using a Debugger

What are the advantages of using a debugger?

Using a debugger like pdb is very helpful for finding bugs in code. You can see the values of all variables and arrays at each step of the program. It lets you go through the code line by line and watch how values change. This makes it easier to find where something goes wrong. You can also check conditions and test different scenarios without changing your code.

Challenges:

No major challenges were encountered. The pdb commands are straightforward once you learn the basic ones like `n` (next), `s` (step), and `p` (print). I am already used to using VS Code breakpoints for debugging, which works similarly.

Note: AI assistance was used for generating plotting code, formatting tables, and print statements.

bonus

February 14, 2026

0.1 Bonus Exercise - Performance Analysis and Optimization of the Game of Life Code

0.1.1 Task B.1

Formatting

[2]:
```
!pylint conway.py
```

```
************* Module conway
conway.py:11:31: C0303: Trailing whitespace (trailing-whitespace)
conway.py:24:38: C0303: Trailing whitespace (trailing-whitespace)
conway.py:25:38: C0303: Trailing whitespace (trailing-whitespace)
conway.py:59:28: C0303: Trailing whitespace (trailing-whitespace)
conway.py:64:70: C0303: Trailing whitespace (trailing-whitespace)
conway.py:66:62: C0303: Trailing whitespace (trailing-whitespace)
conway.py:67:62: C0303: Trailing whitespace (trailing-whitespace)
conway.py:68:74: C0303: Trailing whitespace (trailing-whitespace)
conway.py:95:0: C0303: Trailing whitespace (trailing-whitespace)
conway.py:100:0: C0303: Trailing whitespace (trailing-whitespace)
conway.py:127:18: C0303: Trailing whitespace (trailing-whitespace)
conway.py:80:0: R1707: Disallow trailing comma tuple (trailing-comma-tuple)
conway.py:9:0: C0410: Multiple imports on one line (sys, argparse) (multiple-
imports)
conway.py:12:0: R0402: Use 'from matplotlib import animation' instead (consider-
using-from-import)
conway.py:18:0: C0103: Function name "randomGrid" doesn't conform to snake_case
naming style (invalid-name)
conway.py:18:15: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:22:0: C0103: Function name "addGlider" doesn't conform to snake_case
naming style (invalid-name)
conway.py:29:0: C0103: Function name "addGosperGliderGun" doesn't conform to
snake_case naming style (invalid-name)
conway.py:57:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:57:11: C0103: Argument name "frameNum" doesn't conform to snake_case
naming style (invalid-name)
conway.py:57:32: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
```

```
conway.py:60:4: C0103: Variable name "newGrid" doesn't conform to snake_case
naming style (invalid-name)
conway.py:57:11: W0613: Unused argument 'frameNum' (unused-argument)
conway.py:83:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:97:4: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:99:8: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:102:4: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:104:8: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:9:0: W0611: Unused import sys (unused-import)

------------------------------------
Your code has been rated at 6.18/10
```

[3]:
```
!black conway.py
```

```
reformatted conway.py

All done!
1 file reformatted.
```

[4]:
```
!pylint conway.py
```

```
************* Module conway
conway.py:9:0: C0410: Multiple imports on one line (sys, argparse) (multiple-
imports)
conway.py:12:0: R0402: Use 'from matplotlib import animation' instead (consider-
using-from-import)
conway.py:19:0: C0103: Function name "randomGrid" doesn't conform to snake_case
naming style (invalid-name)
conway.py:19:15: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:24:0: C0103: Function name "addGlider" doesn't conform to snake_case
naming style (invalid-name)
conway.py:30:0: C0103: Function name "addGosperGliderGun" doesn't conform to
snake_case naming style (invalid-name)
conway.py:59:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:59:11: C0103: Argument name "frameNum" doesn't conform to snake_case
naming style (invalid-name)
conway.py:59:32: C0103: Argument name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:62:4: C0103: Variable name "newGrid" doesn't conform to snake_case
naming style (invalid-name)
```

```
conway.py:59:11: W0613: Unused argument 'frameNum' (unused-argument)
conway.py:95:0: C0116: Missing function or method docstring (missing-function-
docstring)
conway.py:111:4: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:113:8: C0103: Variable name "N" doesn't conform to snake_case naming
style (invalid-name)
conway.py:116:4: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:118:8: C0103: Variable name "updateInterval" doesn't conform to
snake_case naming style (invalid-name)
conway.py:9:0: W0611: Unused import sys (unused-import)

------------------------------------------------------------------
Your code has been rated at 7.76/10 (previous run: 6.18/10, +1.58)
```

Documentation

[41]:
```
!mkdir -p docs
!sphinx-quickstart docs -q -p "Bonus" -a "Pavel & Ermia" -v "0" --ext-autodoc
```

```
Finished: An initial directory structure has been created.

You should now populate your master file
/home/pavel/IdeaProjects/DD2358/Ass2/docs/index.rst and create other
documentation
source files. Use the Makefile to build the docs, like so:
   make builder
where "builder" is one of the supported builders, e.g. html, latex or linkcheck.
```

Update `conf.py` with

```
import os
import sys
sys.path.insert(0, os.path.abspath('../..'))
```

to handle being in a separate directory.

[42]:
```
!echo "\nimport os\nimport sys\nsys.path.insert(0, os.path.abspath('../..'))"␣
↪>> docs/conf.py
```

Tell sphinx to autodoc our `conway.py`

[43]:
```
!echo "Conway Module\n=============\n\n.. automodule:: conway\n    :members:\n ␣
↪  :undoc-members:\n    :show-inheritance:" > docs/conway.rst
```

[47]:
```
!echo "   conway" >> docs/index.rst
```

[50]:
```
!make -C docs html
```

```
make: Entering directory '/home/pavel/IdeaProjects/DD2358/Ass2/docs'
Running Sphinx v9.1.0
loading translations [en]… done
loading pickled environment… done
building [mo]: targets for 0 po files that are out of date
writing output…
building [html]: targets for 1 source files that are out of date
updating environment: 0 added, 1 changed, 0 removed
reading sources… [100%] index
looking for now-outdated files… none found
pickling environment… done
checking consistency… done
preparing documents… done
copying assets…
copying static files…
Writing evaluated template result to
/home/pavel/IdeaProjects/DD2358/Ass2/docs/_build/html/_static/basic.css
Writing evaluated template result to /home/pavel/IdeaProjects/DD2358/Ass2/docs/_
build/html/_static/documentation_options.js
Writing evaluated template result to
/home/pavel/IdeaProjects/DD2358/Ass2/docs/_build/html/_static/language_data.js
Writing evaluated template result to
/home/pavel/IdeaProjects/DD2358/Ass2/docs/_build/html/_static/alabaster.css
copying static files: done
copying extra files…
copying extra files: done
copying assets: done
writing output… [100%] index
generating indices… genindex py-modindex done
writing additional pages… search done
dumping search index in English (code: en)… done
dumping object inventory… done
build succeeded.

The HTML pages are in _build/html.
make: Leaving directory '/home/pavel/IdeaProjects/DD2358/Ass2/docs'
```

[51]:
```
from IPython.display import HTML, display

display(HTML(filename="docs/_build/html/conway.html"))
```

```
<IPython.core.display.HTML object>
```

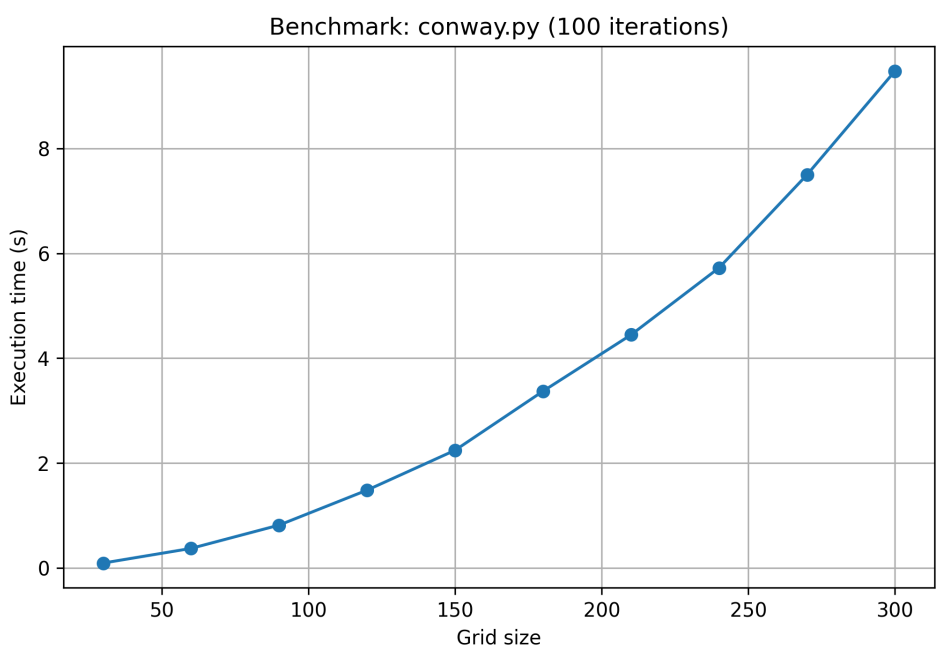
0.1.2 Task B.2

From this point on part the source code was modified to disable the animation and make benchmarking easier.

[53]:
```python
# %load benchmark.py
import conway
import timeit
import matplotlib.pyplot as plt

grid_sizes = [30, 60, 90, 120, 150, 180, 210, 240, 270, 300]

time = []
for size in grid_sizes:
    t1 = timeit.default_timer()
    conway.main(size)
    t2 = timeit.default_timer()
    time.append((t2 - t1))

plt.figure(figsize=(8, 5))
plt.plot(grid_sizes, time, marker='o')
plt.xlabel("Grid size")
plt.ylabel("Execution time (s)")
plt.title(f"Benchmark: conway.py (100 iterations)")
plt.grid(True)
plt.savefig("benchmark.png", dpi=300)
plt.show()
```

[54]:
```python
from IPython.display import Image
Image("benchmark.png")
```

[54]:



As can be seen the growth is quadratic as is the grid size N^2.

0.1.3 Task B.3

[55]:
```
!python -m cProfile -o p.stats conway.py
```

[ ]:
```
import pstats
stats = pstats.Stats("p.stats")
stats.sort_stats("cumulative")
stats.print_stats()
```

Most of the time is spent in the `update` function so gonna focus attention there.

[60]:
```
!python -m kernprof -l -v conway.py
```

```
Wrote profile results to 'conway.py.lprof'
Timer unit: 1e-06 s

Total time: 7.59332 s
File: conway.py
Function: update at line 58

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    58                                           @profile
    59                                           def update(frameNum, img, grid,
N):
    60                                               # copy grid since we
require 8 neighbors for calculation
    61                                               # and we go line by line
    62       100        491.2      4.9      0.0      newGrid = grid.copy()
    63     10100       2758.5      0.3      0.0      for i in range(N):
    64   1010000     276403.1      0.3      3.6          for j in range(N):
    65                                                       # compute 8-neghbor
sum
    66                                                       # using toroidal
boundary conditions - x and y wrap around
    67                                                       # so that the
simulaton takes place on a toroidal surface.
    68   2000000     613692.7      0.3      8.1              total = int(
    69   1000000     312319.6      0.3      4.1                  (
    70   8000000    2722387.2      0.3     35.9                      grid[i, (j
- 1) % N]
    71   1000000     363180.0      0.4      4.8                      + grid[i,
(j + 1) % N]
    72   1000000     386949.0      0.4      5.1                      + grid[(i -
1) % N, j]
    73   1000000     379234.7      0.4      5.0                      + grid[(i +
1) % N, j]
```

```
    74   1000000     396301.1      0.4      5.2                     + grid[(i -
1) % N, (j - 1) % N]
    75   1000000     383924.7      0.4      5.1                     + grid[(i -
1) % N, (j + 1) % N]
    76   1000000     390978.8      0.4      5.1                     + grid[(i +
1) % N, (j - 1) % N]
    77   1000000     383196.9      0.4      5.0                     + grid[(i +
1) % N, (j + 1) % N]
    78                                                             )
    79   1000000     259407.8      0.3      3.4                     / 255
    80                                                         )
    81                                                         # apply Conway's
rules
    82   1000000     413692.2      0.4      5.4                 if grid[i, j] ==
ON:
    83    103960      29584.6      0.3      0.4                     if (total < 2)
or (total > 3):
    84     42425      16544.1      0.4      0.2                         newGrid[i,
j] = OFF
    85                                                         else:
    86    896040     244449.0      0.3      3.2                     if total == 3:
    87     41136      16176.9      0.4      0.2                         newGrid[i,
j] = ON
    88                                                 # update data
    89                                                 #img.set_data(newGrid)
    90       100       1387.6     13.9      0.0      grid[:] = newGrid[:]
    91       100        256.1      2.6      0.0      return (img,)
```

Calculating the neighbor sum takes the majority of the time.

0.1.4 Task B.4

By applying vectorization, using the `roll` function, we can take the neighboring sum outside of the loop and gain performance that way. Next step would be to vectorize the creation of the `newGrid` and apply inplace operations.

[ ]:
```python
def update_old(frameNum, img, grid, N):
    #...
    newGrid = grid.copy()
    for i in range(N):
        for j in range(N):
            total = int(
                (
                        grid[i, (j - 1) % N]
                        + grid[i, (j + 1) % N]
                        + grid[(i - 1) % N, j]
                        + grid[(i + 1) % N, j]
```

```
                        + grid[(i - 1) % N, (j - 1) % N]
                        + grid[(i - 1) % N, (j + 1) % N]
                        + grid[(i + 1) % N, (j - 1) % N]
                        + grid[(i + 1) % N, (j + 1) % N]
                )
                / 255
            )
    #...
```

[ ]:
```
import numpy as np
def update_new(frameNum, img, grid, N):
    #...
    top = np.roll(grid, 1, 0)
    bottom = np.roll(grid, -1, 0)

    total2 = (top + bottom
              + np.roll(grid, 1, 1)
              + np.roll(grid, -1, 1)
              + np.roll(top, 1, 1)
              + np.roll(top, -1, 1)
              + np.roll(bottom, 1, 1)
              + np.roll(bottom, -1, 1)
              ) / 255

    for i in range(N):
        for j in range(N):
            total = total2[i][j]
    #...
```

[ ]:
```
!python -m cProfile conway2.py
```

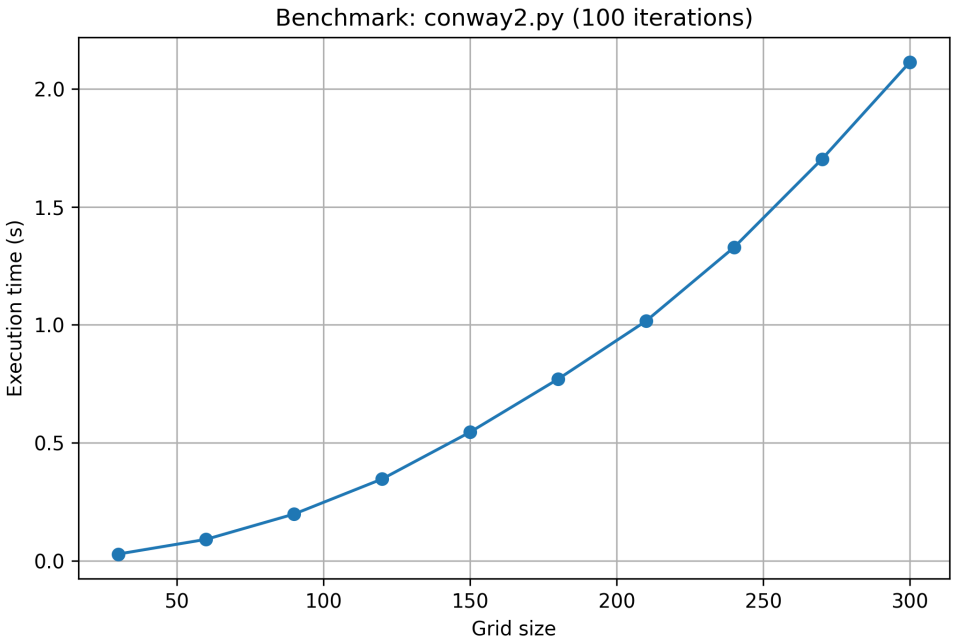
[4]:
```
!python -m kernprof -l -v conway2.py
```

```
Wrote profile results to 'conway2.py.lprof'
Timer unit: 1e-06 s

Total time: 1.4806 s
File: conway2.py
Function: update at line 59

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    59                                           @profile
    60                                           def update(frameNum, img, grid,
N):
    61                                               # copy grid since we
require 8 neighbors for calculation
    62                                               # and we go line by line
```

```
    63       100        433.9      4.3      0.0      newGrid = grid.copy()
    64
    65       100       4443.1     44.4      0.3      top = np.roll(grid, 1, 0)
    66       100       2458.1     24.6      0.2      bottom = np.roll(grid, -1,
0)
    67
    68       800       4572.3      5.7      0.3      total2 = (top + bottom
    69       100       2453.9     24.5      0.2               + np.roll(grid,
1, 1)
    70       100       2140.9     21.4      0.1               + np.roll(grid,
-1, 1)
    71       100       2146.6     21.5      0.1               + np.roll(top, 1,
1)
    72       100       2122.3     21.2      0.1               + np.roll(top,
-1, 1)
    73       100       2095.3     21.0      0.1               + np.roll(bottom,
1, 1)
    74       100       2096.9     21.0      0.1               + np.roll(bottom,
-1, 1)
    75       100         36.8      0.4      0.0               ) / 255
    76
    77     10100       2655.0      0.3      0.2      for i in range(N):
    78   1010000     279656.4      0.3     18.9          for j in range(N):
    79                                                         # compute 8-neghbor
sum
    80                                                         # using toroidal
boundary conditions - x and y wrap around
    81                                                         # so that the
simulaton takes place on a toroidal surface.
    82
    83   1000000     436375.3      0.4     29.5              total =
total2[i][j]
    84                                                         # apply Conway's
rules
    85   1000000     400489.8      0.4     27.0              if grid[i, j] ==
ON:
    86    120129      39637.3      0.3      2.7                  if (total < 2)
or (total > 3):
    87     49651      18532.2      0.4      1.3                      newGrid[i,
j] = OFF
    88                                                         else:
    89    879871     259567.2      0.3     17.5                  if total == 3:
    90     48565      17757.8      0.4      1.2                      newGrid[i,
j] = ON
    91                                                 # update data
    92                                                 #img.set_data(newGrid)
    93       100        759.8      7.6      0.1      grid[:] = newGrid[:]
    94       100        169.8      1.7      0.0      return (img,)
```

[6]:
```python
from IPython.display import Image
Image("benchmark2.png")
```

[6]:



As can be seen the optimization was a success, the correctness was tested with `assert` against the original implementation. The performance gains are significant as now we can run a 300x300 grind in about the same time it took for a 150x150 grid with the old implementation.